SkillForge AI — Master Project Report

> Enterprise AI-powered Learning Management System (LMS) Report Date: 2026-05-28

Table of Contents

1. Project Overview

SkillForge AI is a full-stack enterprise Learning Management System built for organizations to assign, track, and assess employee training at scale. It combines a role-aware React frontend with an Express REST API backend, a dual-mode database layer (Supabase PostgreSQL in production, JSON file fallback in development), and an AI layer that dynamically generates quiz questions for each learning session.

Core Value Propositions

Capability	Description
AI-generated assessments	10 unique quiz questions generated per session via LLM
Role-scoped dashboards	Separate UX for admin, manager, and employee roles
Session progress persistence	Granular per-session completion tracking with score history
Manager-scoped reports	Managers see only their team; admins see the entire platform
Enterprise table UI	Hover-reveal actions, skeleton loaders, live refresh, record counts
Client-side rate limiting	Login lockout persisted across page refreshes via localStorage

Project Scale

- 5 admin pages, 1 manager dashboard, employee module flow (4 views), shared auth/report pages
- 15+ API route groups
- 3 database entity types: users, assignments, modules
- 3 RBAC roles with fine-grained permission enforcement on both client and server

---

2. Technology Stack

Frontend

Layer	Technology	Version / Notes
Framework	React	18
Build tool	Vite	Fast HMR, ES module bundling
Styling	Tailwind CSS	JIT mode; static class names required to avoid purge
Routing	React Router	Client-side SPA routing
State	React Context API	AuthContext for JWT + role
HTTP	fetch API	Native, no Axios

Backend

Layer	Technology	Notes
Runtime	Node.js	ES Modules ("type": "module")
Framework	Express	REST API
Auth	JWT (jsonwebtoken)	HS256, stored in localStorage client-side
Password hashing	bcrypt	Server-side hash comparison
Rate limiting	express-rate-limit	5 attempts / 15 min on login
Module system	ES Modules	import/export throughout, no CommonJS

Database

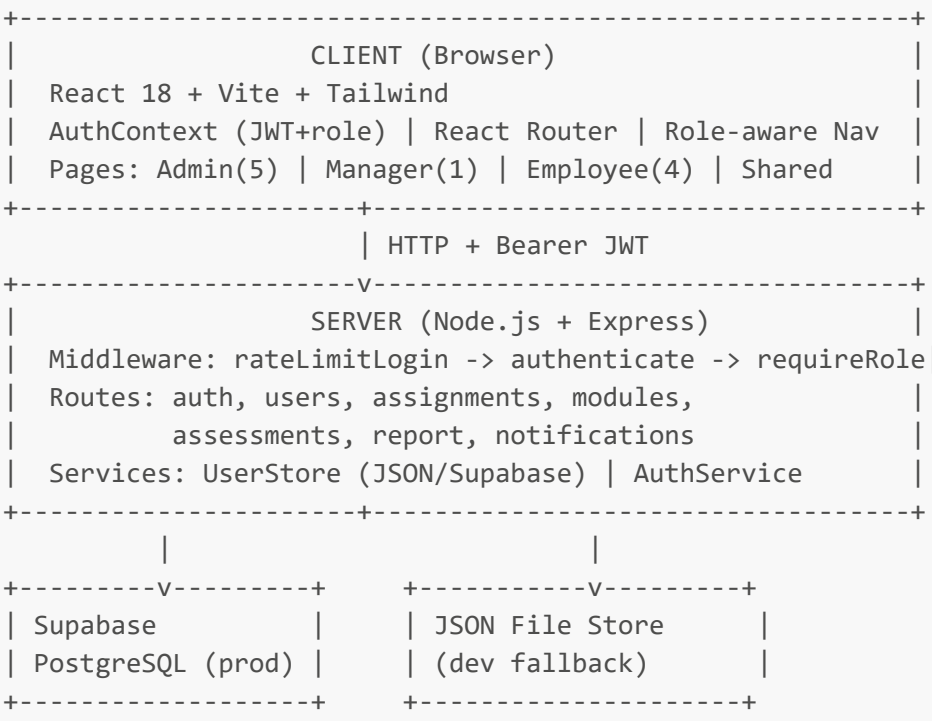
Mode	Technology	When Used
Production	Supabase PostgreSQL	Cloud, persistent
Development	JSON file store	Local fallback, no cloud dependency

AI Layer

- Endpoint: POST /api/assessments/generate
- Generates 10 multiple-choice questions per session based on session content
- Questions are stateless per request (no caching — fresh questions on each Retry)

---

3. System Architecture



Session Completion Data Flow

```
Employee submits quiz
  -> QuizPhase calls onComplete(scoreData)
  -> ModuleSession: GET /api/assignments/:id  (fetch current)
  -> Merge: { ...existingSessionProgress, [sessionIndex]: 'completed' }
  -> PUT /api/assignments/:id
  -> Server safety merge: { ...existingProgress, ...incomingProgress }
  -> ScorePhase renders with 4 action buttons
```

---

4. Authentication & Authorization

Auth Flow

```
1. POST /api/auth/login (rate-limited: 5 req / 15 min)
2. Server: bcrypt.compare(password, hash)
3. Server: jwt.sign({ userId, role, name }, SECRET, { expiresIn: '24h' })
4. Client: stores token in localStorage
5. Client: sends Authorization: Bearer <token> on every request
6. Server middleware: jwt.verify(token) -> req.user = { userId, role, name }
7. requireRole(['admin']) -> 403 if role mismatch
```

Middleware Chain

```
router.get('/admin-only', authenticate, requireRole(['admin']), handler)
router.get('/manager-admin', authenticate, requireRole(['admin', 'manager']),
handler)
router.get('/any-role', authenticate, handler)
```

Client-Side Rate Limit Lockout

```
RATE_LIMIT_KEY = 'skillforge:login_locked_until'

On 5th failed attempt:
  lockoutUntil = Date.now() + 15 * 60 * 1000
  localStorage.setItem(RATE_LIMIT_KEY, lockoutUntil)
  setInterval(tick, 1000) -> countdown display

On page load:
  Check localStorage -> if future timestamp -> restore lockout state

On successful login:
  localStorage.removeItem(RATE_LIMIT_KEY)
  failedAttempts = 0
```

Warning Progression

Failed Attempts	UI State
1-2	Silent
3	"2 attempts remaining before lockout"
4	"1 attempt remaining before lockout"
5	Lockout banner + countdown, form disabled
Server 429	Lockout triggered from server response

---

5. Database Design

users

Field	Type	Notes
userId	string	UUID
name	string	Display name
email	string	Unique, used for login
role	enum	admin / manager / employee
passwordHash	string	bcrypt hash

Field	Type	Notes
emailVerified	boolean	
managerId	string/null	References userId of assigned manager
createdAt	ISO timestamp	

assignments

Field	Type	Notes
id	string	UUID
type	string	e.g. "module"
assignable_id	string	References modules.id
assigned_to_user	string	References users.userId
assigned_by	string	Creator userId
assigned_by_manager	string	Manager userId
status	enum	assigned / in_progress / completed / overdue
progress	number	0-100 overall percentage
sessionProgress	object	{ "0": "completed", "1": "completed" }
progress_data	object	{ sessionReports: { "0": { score, correct, total, completedAt, strengths[], weaknesses[] } } }
dueDate	ISO timestamp	
priority	enum	low / medium / high
created_at	ISO timestamp	
updated_at	ISO timestamp	

modules

Field	Type	Notes
id	string	UUID
title	string	Module display name
description	string	Summary
difficulty	enum	beginner / intermediate / advanced
duration	number	Minutes

Field	Type	Notes
skills	string[]	Skill tags
sessions	object[]	Array of session definitions
content	object	AI-generated content
roadmap	object	Learning path metadata

Relationships

```
users (manager) --< users (employee)  [managerId FK]
users --< assignments                 [assigned_to_user FK]
modules --< assignments               [assignable_id FK]
```

---

6. Frontend Structure & Routing

Route Map

Path	Component	Access
/auth/login	Login.jsx	Public
/dashboard	DashboardRedirect	Authenticated
/employee/dashboard	Employee.jsx	employee
/admin/dashboard	AdminDashboard.jsx	admin
/admin/users	UserManagement.jsx	admin
/admin/modules	ModuleManagement.jsx	admin
/admin/assignments	AssignmentManagement.jsx	admin
/admin/assessments	AssessmentManagement.jsx	admin
/manager/dashboard	ManagerDashboard.jsx	manager
/module/:moduleId/start	ModuleStart.jsx	employee
/module/:moduleId/learn	ModuleDashboard.jsx	employee
/module/:moduleId/session/:sessionIndex	ModuleSession.jsx	employee
/report	Report.jsx	admin, manager

DashboardRedirect Logic

```
/dashboard ->
  role === 'admin'    -> /admin/dashboard
```

```
role === 'manager'  -> /manager/dashboard
role === 'employee' -> /employee/dashboard
```

Navbar Role Links

Role	Links Shown
admin	Dashboard, Users, Modules, Assignments, Assessments
manager	Dashboard, Report
employee	My Learning (/dashboard)

Active State Fix (Navbar)

The employee "My Learning" link points to /dashboard but resolves to /employee/dashboard. Fixed with:

```
isActive =
  location.pathname === to ||
  (location.pathname === '/employee/dashboard' && to === '/dashboard') ||
  (location.pathname.startsWith(to) && to !== '/manager/dashboard')
```

---

7. Backend API Reference

Auth

Method	Path	Auth	Notes
POST	/api/auth/login	None	Rate-limited 5/15min
POST	/api/auth/logout	JWT	Token invalidation

Users

Method	Path	Auth	Notes
GET	/api/users	admin	List all, filter by ?role=
POST	/api/users	admin	Create user
PUT	/api/users/:id	admin	Update user
DELETE	/api/users/:id	admin	Delete user

Assignments

Method	Path	Auth	Notes
GET	/api/assignments	JWT	Manager-scoped server-side
POST	/api/assignments	admin, manager	Create assignment
GET	/api/assignments/:id	JWT	Single assignment
PUT	/api/assignments/:id	JWT	Server merges sessionProgress
DELETE	/api/assignments/:id	admin, manager	
GET	/api/assignments/manager/:managerId/employees	manager, admin	
GET	/api/assignments/employee/:userId/manager	JWT	Get manager for employee
POST	/api/assignments/content	JWT	Content fetch

Modules

Method	Path	Auth	Notes
GET	/api/modules	JWT	List all modules
GET	/api/modules/:id	JWT	Single module with sessions
POST	/api/modules	admin	Create module
PUT	/api/modules/:id	admin	Update module
DELETE	/api/modules/:id	admin	Delete module

Assessments

Method	Path	Auth	Notes
POST	/api/assessments/generate	JWT	AI question generation
GET	/api/assessments	admin	List assessments

Note: Server mounted at both /api/assessment and /api/assessments (bug fix).

Report

Method	Path	Auth	Role Scoping
GET	/api/report/all	JWT	admin=all; manager=own team only

Server-Side sessionProgress Merge Pattern

```
const existing = await AssignmentStore.getById(id)
const existingProgress = existing.sessionProgress || {}
updates.sessionProgress = { ...existingProgress, ...sessionProgress }
```

---

8. Role-Based Feature Matrix

Feature	admin	manager	employee
View all users	Yes	No	No
Create/edit/delete users	Yes	No	No
Assign manager to employee	Yes	No	No
View all modules	Yes	Yes (via assignment)	Yes (assigned only)
Create/edit modules	Yes	No	No
Create assignments	Yes	Yes (own team)	No
View assignments	All	Own team only	Own only
View reports	All employees	Own team only	No
Generate AI assessments	Yes	No	No
Complete sessions/quizzes	No	No	Yes
View own learning progress	No	No	Yes
Auto-refresh user table	Yes	N/A	N/A
Login rate limiting	All	All	All

---

9. AI Integration

Quiz Generation Flow

```
1. Employee reaches end of session content
2. Client: POST /api/assessments/generate
   Body: { moduleId, sessionIndex, sessionContent, topic }
3. Server: calls LLM with prompt including session content
4. LLM returns: 10 multiple-choice questions with correct answers
5. Client: renders QuizPhase component with questions
6. Employee answers -> score calculated client-side
7. Score + metadata saved to assignment.progress_data.sessionReports
```

Question Format

```
{
  "question": "What is the primary purpose of useEffect in React?",
  "options": ["A) ...", "B) ...", "C) ...", "D) ..."],
  "correct": "B",
  "explanation": "..."
}
```

Retry Mechanism (quizKey Pattern)

```
const [quizKey, setQuizKey] = useState(0)
// On retry — forces full component remount, fresh AI fetch:
setQuizKey(k => k + 1)
// In JSX:
<QuizPhase key={quizKey} {...props} />
```

Score Storage (per session)

```
{
  "score": 80,
  "correct": 8,
  "total": 10,
  "completedAt": "2026-05-28T10:30:00Z",
  "strengths": ["topic A", "topic B"],
  "weaknesses": ["topic C"]
}
```

---

10. Key Bugs Found & Fixed

Bug 1 — sessionProgress Replacement (Critical Data Loss)

Symptom: Completing session 3 erased completion records for sessions 0, 1, 2.

Root Cause (client): Sent only { [sessionIndex]: 'completed' } — a single-key object. Server assigned it directly, replacing all prior keys.

Root Cause (server): No server-side merge protection.

Fix (client): Fetch current assignment first, merge, then PUT full merged object.

Fix (server): `updates.sessionProgress = { ...existingProgress, ...sessionProgress }`

Bug 2 — Score Gate + Undefined Score (Progress Never Saved)

Symptom: Quiz completion never saved to the database.

Root Cause 1: `if (score?.pct >= 70)` gate — blocked saves below threshold.

Root Cause 2: ScorePhase rendered inside QuizPhase called `onComplete()` with no arguments — parent received `undefined` as score.

Fix: Removed score gate. Lifted ScorePhase to top-level ModuleSession. QuizPhase now calls `onComplete(scoreData)` with actual data.

Bug 3 — Session Status Init Wipe

Symptom: Returning to module dashboard reset ALL session statuses to `{ 0: 'pending' }`.

Root Cause: Two sequential setState calls — second overwrote first.

```
// Bug:
setSessionStatuses(progress)
if (!progress[0]) setSessionStatuses({ 0: 'pending' })  // wipes all
```

Fix: Single merged initialization:

```
const initialStatuses = progress[0] !== undefined
  ? progress
  : { ...progress, 0: 'pending' }
setSessionStatuses(initialStatuses)
```

Bug 4 — Dynamic Tailwind Class Purging

Symptom: Color badges showed no color in production build.

Root Cause: `text-${color}-400` template literals not scanned by Tailwind JIT.

Fix: Static class lookup map — all class strings present as literals in source.

Bug 5 — Assessment 404

Symptom: Client assessment requests returned 404.

Root Cause: Server at `/api/assessment` (singular), client called `/api/assessments` (plural).

Fix: Mount router at both paths.

Bug 6 — Manager Seeing All Employees

Symptom: Manager dashboard showed all platform employees.

Root Cause: Fallback code called `GET /api/users?role=employee` (platform-wide) when no assigned employees found.

Fix: Removed fallback. Show empty state instead.

Bug 7 — Manager Reports Unfiltered

Symptom: `/api/report/all` returned all employees regardless of requester role.

Fix: Server-side role check — admin gets all, manager gets `getManagerEmployees(req.user.userId)`.

Bug 8 — Navbar Active State Not Highlighting

Symptom: "My Learning" link never showed active state on employee dashboard.

Root Cause: Link is `/dashboard`, resolved path is `/employee/dashboard`. Equality check fails.

Fix: Added explicit case: `location.pathname === '/employee/dashboard' && to === '/dashboard'`

Bug 9 — Dead ScorePhase Callback

Symptom: `handleQuizComplete(score)` always received `undefined`.

Root Cause: ScorePhase nested inside QuizPhase called `onComplete()` with no args.

Fix: Moved ScorePhase to top-level. QuizPhase calls `onComplete(scoreData)`.

Bug 10 — Login Lockout Not Persisted

Symptom: Page refresh reset failed attempt counter, bypassing rate limiting.

Fix: Persist lockout timestamp to localStorage. Check on component mount. Clear on success.

---

11. UI/UX Engineering Decisions

Enterprise Table Design System

All 5 admin tables use a unified visual language:

Element	Implementation
Container	`bg-[#111827]` + `shadow-xl` (hardcoded hex avoids Tailwind purge)
Layout	CSS Grid via `style={{ gridTemplateColumns: '...' }}`
Hover actions	`group` on row + `opacity-0 group-hover:opacity-100` on buttons
Loading	Animated skeleton rows with `animate-pulse`
Footer	Record count + "Last refreshed HH:MM:SS" + Refresh button

Why CSS Grid over HTML table?

- Precise column widths without table's implicit sizing algorithm
- Works cleanly with Tailwind utility classes
- Easier hover state + row selection patterns
- Avoids `table-*` Tailwind utilities (purge-prone)
- Trade-off: must add ARIA `role="table"` for accessibility

Skeleton Loading Pattern

```
{loading && Array.from({ length: 5 }).map((_, i) => (
  <div key={i} className="animate-pulse bg-slate-700/50 h-12 rounded" />
))}
```

ScorePhase Action Buttons (4 options)

Button	Action
Rewatch Session	Back to ContentPhase
Retry Quiz	quizKey++ -> remount QuizPhase -> fresh AI questions
Continue to Day N+1	Navigate to session index+1
Back to Module Dashboard	Navigate to /module/:id/learn

Progress Saving UX Indicator

```
Submit quiz -> "Saving progress..." -> PUT completes -> "Progress saved" ->
ScorePhase
```

AssignmentManagement Pending Card

Clickable stat card opens modal listing all employees with no assignment — lets admin identify and close gaps quickly.

UserManagement Auto-Refresh

```
useEffect(() => {
  fetchData()
  const interval = setInterval(fetchData, 30_000)
  return () => clearInterval(interval)
}, [])
```

---

12. Security Implementation

Authentication Security

Concern	Implementation
Password storage	bcrypt hashing — never plaintext
Token expiry	JWT `expiresIn: '24h'`
Token transport	Authorization: Bearer header
Token storage	localStorage (XSS trade-off; acceptable for internal tool)

Rate Limiting (Two Layers)

Layer	Implementation
Server	express-rate-limit: 5 req / 15 min on /api/auth/login, by IP
Client	localStorage-persisted lockout timestamp, survives page refresh
UX	Countdown timer, progressive warnings, form disabled

RBAC Enforcement

- requireRole([...]) middleware enforces roles server-side on every request
- Client-side role checks are UX-only — UI hiding, not security
- Server always re-validates role from JWT regardless of client state
- Manager data scoping enforced server-side (client cannot override)

Security Incident

- Hardcoded Supabase service key found in server/db/_probe2.js and supabaseinfo.*
- Files added to .gitignore; never committed to version control
- Action required: rotate Supabase service key immediately in dashboard
- Lesson: secrets belong in .env files, never in source code

---

13. Design Patterns & Reusable Solutions

1. Server-Side Safety Merge

```
updates.sessionProgress = {
  ...existingRecord.sessionProgress,
  ...incomingSessionProgress
}
```

Use for any partial update to a nested object where existing data must be preserved.

2. Component Remount via Key Increment

```
const [quizKey, setQuizKey] = useState(0)
setQuizKey(k => k + 1)  // trigger remount
<QuizPhase key={quizKey} {...props} />
```

Use when you need completely fresh component state (not just re-render).

3. Promise.allSettled for Parallel Requests

```
const [r1, r2, r3] = await Promise.allSettled([
  fetch('/api/users'),
  fetch('/api/assignments'),
  fetch('/api/modules'),
])
// Each: { status: 'fulfilled'|'rejected', value|reason }
```

Use for dashboard multi-source loads — one failure does not block the rest.

4. Polling with useEffect Cleanup

```
useEffect(() => {
  fetchData()
  const interval = setInterval(fetchData, 30_000)
  return () => clearInterval(interval)
}, [])
```

5. localStorage Rate Limit Persistence

```
const RATE_LIMIT_KEY = 'skillforge:login_locked_until'
localStorage.setItem(RATE_LIMIT_KEY, (Date.now() + 15*60*1000).toString())
// On mount: check if timestamp is in future -> restore lockout
// On success: localStorage.removeItem(RATE_LIMIT_KEY)
```

6. Hover-Reveal Action Pattern

```
<div className="group ...">
  {/* row content */}
  <div className="opacity-0 group-hover:opacity-100 transition-opacity duration-
150">
    <button>Edit</button>
    <button>Delete</button>
  </div>
</div>
```

7. Tailwind Static Class Map (Purge-Safe)

```
// NEVER: `text-${color}-400`
const STATUS_COLORS = {
  completed:   'text-green-400 bg-green-400/10',
  in_progress: 'text-yellow-400 bg-yellow-400/10',
  overdue:     'text-red-400 bg-red-400/10',
  assigned:    'text-blue-400 bg-blue-400/10',
}
```

8. EditModal Manager Persistence

```
// On modal open — pre-fetch current manager assignment:
const res = await fetch(`/api/assignments/employee/${user.userId}/manager`)
const { managerId } = await res.json()
setCurrentManagerId(managerId)
// Show "(current)" in dropdown, disable Assign button if same manager selected
```

---

14. Session Progress Flow (End-to-End)

Complete Flow

```
Employee clicks module
  -> ModuleStart.jsx (overview + "Start Learning")
  -> ModuleDashboard.jsx /module/:id/learn
       SessionsTab: sessions list with status badges
       ProgressTab: scores, dates, strengths/weaknesses, avg score
       Init: single merged setSessionStatuses call
  -> Employee clicks session
  -> ModuleSession.jsx /module/:id/session/:index

     Phase 1: ContentPhase
       Renders session content
       "Take Quiz" button

     Phase 2: QuizPhase  [key={quizKey}]
       POST /api/assessments/generate -> 10 AI questions
       Employee submits answers
       Score calculated client-side
       onComplete(scoreData) called

     handleQuizComplete(scoreData):
       1. Show "Saving progress..."
       2. GET /api/assignments/:id  (fetch current)
       3. Merge: { ...existing.sessionProgress, [idx]: 'completed' }
       4. PUT /api/assignments/:id {
            sessionProgress: mergedProgress,
```

```
            progress_data: { sessionReports: { [idx]: { score, correct, total,
completedAt, strengths, weaknesses } } }
          }
       5. Server safety merge
       6. Show "Progress saved"

     Phase 3: ScorePhase
       Shows score, correct/total, strengths, weaknesses
       Rewatch -> ContentPhase
       Retry   -> quizKey++, fresh QuizPhase
       Continue -> session idx+1
       Dashboard -> /module/:id/learn
```

Session Status State Machine

```
Session 0: pending -> in_progress -> completed
Session 1: locked until 0 completed -> pending -> ...
Session N: locked until N-1 completed
```

ProgressTab Data

Per completed session: score %, correct/total, completion date, strengths (green), weaknesses (yellow), average score across all sessions.

---

15. Interview Prep: Questions & Answers

Q: Why JWT in localStorage instead of httpOnly cookies?

A: localStorage is simpler to implement in a SPA with a separate server origin. The trade-off is XSS risk — malicious scripts can read localStorage. HttpOnly cookies prevent JavaScript access entirely and are more secure. For this internal enterprise tool, localStorage was the acceptable pragmatic choice. Production-grade systems with sensitive data should use httpOnly cookies with SameSite=Strict and CSRF tokens.

Q: How does RBAC work in this system?

A: Three layers: (1) JWT payload carries the user's role, set at login and tamper-evident. (2) Server middleware requireRole(['admin']) rejects requests with wrong roles — 403. (3) Client-side role checks hide irrelevant UI. Only layers 1 and 2 are security controls. Layer 3 is UX only. Key example: manager calling /api/report/all still gets only their team because the server enforces scoping via getManagerEmployees(req.user.userId), regardless of what the client requests.

Q: Explain the sessionProgress replacement bug and how you fixed it.

A: Two independent failures combined. Client-side: we sent only { [sessionIndex]: 'completed' } — one key — instead of merging with existing progress. Server-side: we assigned that partial object directly, overwriting all prior keys. Fix: client fetches the current assignment first, spreads existing progress, adds the new key, sends

the full merged object. Server adds a redundant merge as defensive programming. This pattern — fetch, merge, put — is the correct approach for any partial update to a nested field when the API uses PUT semantics.

Q: What is the quizKey pattern and why is it needed?

A: React re-renders when state changes but only unmounts+remounts when the key prop changes. On quiz retry, we need completely fresh state — fresh AI questions, cleared answers, reset timer. Incrementing quizKey causes React to destroy the old QuizPhase instance and mount a new one, which triggers a new POST /api/assessments/generate. This is cleaner than manually resetting every piece of state inside the component.

Q: Why CSS Grid instead of HTML table for admin tables?

A: CSS Grid gives explicit, predictable column widths via gridTemplateColumns without the implicit sizing behavior of HTML tables. It integrates cleanly with Tailwind utilities and makes hover state rows (group/group-hover) straightforward. The trade-off is losing native table accessibility semantics, addressable with role="table", role="row", role="cell" ARIA attributes.

Q: How did you fix the dynamic Tailwind class purging issue?

A: Tailwind's JIT compiler scans source files for complete class name strings. Template literals like text-${color}-400 are not recognized as class names — the scanner sees a JavaScript expression, not a CSS class. The fix is a static lookup object where every possible class name exists as a literal string in the source code. JIT then includes all of them in the output bundle.

Q: How would you identify the sessionProgress replacement bug in a code review?

A: Look for PUT/PATCH handlers that accept nested objects. Trace: does the handler fetch the current record before writing? Does it merge or replace? If the incoming body contains a subset of the full nested object and the handler assigns it directly, data loss will occur. The signal here was the client sending a single-key object { [sessionIndex]: 'completed' } to a field that can contain many keys.

Q: Why did progress never save even after removing the score gate?

A: Two independent bugs that both had to be present for progress to fail. First: score gate if (score?.pct >= 70) blocked low scores. Second: score was always undefined because ScorePhase called onComplete() with no arguments — the score data never traveled up to the parent handler. Both bugs needed fixing independently. Removing only the gate still left undefined flowing in; score?.pct on undefined is still undefined, and undefined >= 70 is false.

Q: How does the client-side rate limiting work, and is it sufficient for security?

A: Client tracks failed attempts and persists a lockout timestamp to localStorage. On page load, if the timestamp is in the future, it restores the lockout UI — countdown timer, disabled form. This is UX, not security. A determined user can open devtools, clear localStorage, and immediately retry. The true security

control is express-rate-limit on the server, which enforces by IP address and cannot be bypassed by any client-side manipulation.

Q: How does manager report scoping work at the API level?

A: The JWT contains the user's userId and role. In /api/report/all, the handler checks req.user.role. Admins get UserStore.getAll({ role: 'employee' }). Managers get UserStore.getManagerEmployees(req.user.userId), which filters users by their managerId field. The manager's userId comes from the verified JWT — they cannot claim to be a different manager by modifying the request body.

Q: What is the fetch-merge-PUT pattern and when should you use it?

A: When a PUT endpoint replaces the entire resource (not PATCH/merge-patch), and your client only has partial data to update, you must first GET the full current state, merge your changes, then PUT the complete object. Required when: (1) API lacks PATCH support, (2) multiple concurrent clients may update the same resource, (3) client only knows the new value for a subset of fields. For concurrent safety, the server should also apply a merge as a redundant safeguard — two-layer protection.

Q: What dead code was removed and why does it matter?

A: Employee.jsx had openModulePlan, selectedModule, moduleContent, showPlanModal — state variables initialized but never connected to any JSX or event handler. ModuleDashboard.jsx had handleSessionComplete function and onSessionComplete prop that were declared but never called. Dead code increases cognitive load, misleads future developers about system behavior, and can mask real bugs (a handler that "should" be called but isn't). Cleanup was done after confirming no callers existed in either direction.

End of SkillForge AI Master Project Report